# SimQC – Weekly Progress Report

Aoutul Nabi Purna*, Shishir Dhar†, Shihab Sharar‡, and Md. Iftikhar Alam Hridik§
Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh
Emails: {*aoutul.purna.241, †shishir.dhar.241, ‡shihab.sharar.241, §iftikhar.hridik.241}@northsouth.edu

***Abstract*—This report outlines Sprint 2 of the SimQC quantum simulator, covering the state-vector engine core, the matrix-free pairwise gate-application kernel, the gate/parameter/registry layer, the circuit model with subcircuit inlining and flattening, the canonical JSON serializer, and the differential test harness validating the $O(2^n)$ kernel against the $O(4^n)$ Kronecker reference. The sprint translates the frozen conventions from Sprint 1 into a working zero-allocation simulation engine in JavaScript, with differential validation against the naive reference implementation for circuits up to six qubits.**

## I. Progress Summary

During this sprint, we implemented the simulation core specified in the frozen conventions from Sprint 1[cite: 1]. Completed deliverables include:

1) A `StateVector` class exposing a clean facade (`get`/`set`, `probability`, `probabilities`, `normSquared`, `clone`, `renormalize`) while storing amplitudes in two flat `Float64Array` buffers (`re`, `im`), initialized to the zero state $\alpha_0 = 1$, all other amplitudes zero[cite: 1]. Internal buffer references `_re` and `_im` are exposed strictly for kernel access, preserving encapsulation for external consumers[cite: 1].
2) A zero-allocation pairwise kernel in `kernel.js`. The `apply1QubitOp` function uses stride-$2^t$ block iteration over bitwise index pairs $(i, i \oplus (1 \ll t))$, with bitwise control masks `(i & controlMask) === controlMask` and anti-control filtering. The inner loop unrolls the canonical $2 \times 2$ complex multiply-add on the flat 8-double tuple $g = [g_{00}^r, g_{00}^i, g_{01}^r, g_{01}^i, g_{10}^r, g_{10}^i, g_{11}^r, g_{11}^i]$, operating exclusively on primitive doubles with zero heap allocation[cite: 1]. A separate `applySwap` function implements SWAP/CNOT permutations via direct amplitude exchange without floating-point multiplication[cite: 1].
3) A gate layer comprising `basis.js`, `param.js`, and `registry.js`. `basis.js` defines static gates $(I, X, Y, Z, H, S, T)$ using Sprint 1's `Matrix` class for readability, then flattens each to an 8-tuple via `matrixToFlat8` at module initialization. `param.js` implements parameterized rotations $(R_x, R_y, R_z)$ as pure functions of $\theta$ returning flat tuples directly. `registry.js` exposes an immutable map resolving string identifiers (`"H"`, `"X"`, `"Rx"`) to flat tuples or generator functions, with a debug assertion verifying unitarity $\|G^\dagger G - I\| < 10^{-12}$ on registration[cite: 1].
4) A circuit model comprising `GateColumn`, `Circuit`, `Flatten`, and `Ascii`. `GateColumn` is a passive value class representing a time slice across qubit rows, with intra-column disjointness assertions preventing two operations from sharing target or control qubits. `Circuit` is an immutable list of columns with `withColumnAt`, `withWires`, and `numQubits`. `Flatten` implements a dual-layer pipeline: Layer 1 recursively inlines subcircuits into flat instruction objects carrying `virtualColumn` metadata; Layer 2 groups instructions sharing a virtual column into contiguous execution batches. `Ascii` ports Quirk's diagram parser to generate test circuits from text diagrams[cite: 1].
5) A canonical JSON `Serializer` (`toJson`/`fromJson`) adhering to the v1 Canonical Schema: explicit `"format": "simqc-circuit"`, `"version": 1`, `"bitOrder": "little-endian"`, with control lines as explicit objects `controls: [wire: 0, on: true]`[cite: 1].
6) A differential test harness (`kernel.test.js`, `Circuit.test.js`, `GateColumn.test.js`, `Serializer.test.js`) comparing the pairwise kernel against Sprint 1's $O(4^n)$ Kronecker reference over pseudo-random circuits for $n \leq 6$, asserting norm preservation $\|\psi\|^2 \in 1.0 \pm 10^{-9}$ after each gate[cite: 1].

The repository structure was finalized under `src/core`, `src/circuit`, `src/gates`, and `src/cli`, with `CONVENTIONS.md`, `FOLDER_STRUCTURE.md`, and `SOURCE_OF_TRUTH.md` frozen as the source of truth[cite: 1].

## II. Learning: Matrix-Free Pairwise Simulation vs. Kronecker Products

The textbook Kronecker product approach lifts a single-qubit gate $G$ on target qubit $t$ to the full $2^n \times 2^n$ operator:

$$U = I \otimes \cdots \otimes G \otimes \cdots \otimes I, \tag{1}$$

then multiplies $U$ against the state vector at $\mathcal{O}(4^n)$ cost[cite: 1]. At 16 qubits, a single lifted operator matrix requires ∼34 GB of RAM, and the matrix is overwhelmingly sparse—for $(I \otimes I \otimes H)$ on 3 qubits, 48 of the 64 entries are zero, meaning 75% of the arithmetic operates on zero amplitudes[cite: 1].

The matrix-free pairwise method instead updates only the $2^{n-1}$ amplitude pairs whose indices differ in bit $t$ ($j = i \oplus 2^t$), each with the same $2 \times 2$ matrix $G$, yielding $\mathcal{O}(2^n)$ runtime per gate[cite: 1]. Our worked 3-qubit example confirmed that

the pairwise method and the naive Kronecker product are numerically identical, and that CNOT reduces to pure in-place amplitude swapping (a permutation) requiring no floating-point multiplication[cite: 1].

**Entanglement forbids lazy Kronecker products.** We considered keeping each qubit as an independent $2 \times 1$ vector and deferring the tensor product to the end[cite: 1]. This holds only while the state is separable: the 2-qubit separability condition $c_{00}c_{11} - c_{01}c_{10} = 0$ holds after single-qubit gates but is violated the moment an entangling gate (CNOT, CZ) acts, since CNOT does not factorize as $A \otimes B$[cite: 1]. Simulators must therefore maintain the full $2^n$-dimensional vector from the start[cite: 1].

**Object allocation does not belong in the hot path.** Although Sprint 1's `Complex` class is complete and correct, representing the state vector as an array of `Complex` objects was rejected[cite: 1]. In V8, each object overhead costs roughly 40–56 bytes versus 16 bytes for two 64-bit floats in a `Float64Array` ($\approx 3\times$ memory reduction)[cite: 1]. Complex multiply-add with objects allocates temporary heap instances; at $2^{20}$ amplitudes per gate this triggers garbage-collector pauses that freeze the UI during drag-and-drop[cite: 1]. The kernel therefore operates exclusively on primitive doubles with zero heap allocation[cite: 1].

## III. Architecture Decisions

- **Layout:** little-endian indexing ($j = i \oplus (1 \ll q)$); two flat `Float64Array` buffers behind the `StateVector` facade[cite: 1].
- **Kernel arithmetic:** raw scalar doubles only; no `Complex` instances in gate-application loops[cite: 1].
- **Controlled gates:** pair update applied only when `(i & controlMask) === controlMask`; non-matching amplitudes left untouched bit-for-bit[cite: 1].
- **Normalization:** norm verified after every gate; renormalize only if drift $> 10^{-9}$, since silent corrections mask arithmetic bugs[cite: 1].
- **Column semantics:** gates within one circuit column act on disjoint qubits; overlap prevented at construction[cite: 1].
- **Serialization:** explicit qubit indices; little-endian declared in schema metadata[cite: 1].

## IV. Differential Validation

The harness compares the $O(2^n)$ pairwise kernel against Sprint 1's $O(4^n)$ Kronecker reference over pseudo-random circuits for $n \leq 6$, requiring agreement within $\epsilon \leq 10^{-12}$ and norm $\|\psi\|^2 \in 1.0 \pm 10^{-9}$ after each gate[cite: 1]. Bidirectional CNOT tests ($0 \to 1$ swaps $\{1,3\},\{5,7\}$; $1 \to 0$ swaps $\{2,3\},\{6,7\}$) guard against endianness slips[cite: 1]. The Bell-state exit criterion ($H$ then CNOT$(0 \to 1)$ yields $(0.7071, 0, 0, 0.7071)$) is verified end-to-end through the headless CLI: `cat test/fixtures/bell_state.json | node src/cli.js`[cite: 1]. Agreement between the naive Kronecker path and the pairwise kernel is maintained within tolerance across all tested depths and gate mixtures[cite: 1].

## V. Next Steps

1) Complete the headless CLI runner (`src/cli.js`) consuming canonical circuit JSON from `stdin` and outputting state-vector amplitudes or probabilities to `stdout`[cite: 1].
2) Extend the differential harness to cover parameterized gates ($R_x, R_y, R_z$) and multi-qubit controlled operations[cite: 1].
3) Begin Phase 1 of the AI module: local inference via Ollama/llama.cpp under the frozen `AI_INTERFACE.md` contract, with `qwen2.5:3b` as the default development model[cite: 1].
4) Explore multi-threading (Worker threads) for the kernel's block iteration to parallelize independent amplitude pairs[cite: 1].

**Exit criterion:** differential validation passes for $n \leq 6$ at $\epsilon \leq 10^{-12}$, and the CLI produces Bell-state probabilities $[0.5, 0.0, 0.0, 0.5]$ from `bell_state.json`[cite: 1].

## VI. Risks and Mitigations

1) **Memory ceiling:** $\sim$256 MB at 24 qubits; mitigated by enforcing a hard simulation limit at 24 qubits with UI threshold warnings at 20 qubits[cite: 1].
2) **GC pressure:** mitigated by the zero-allocation kernel design detailed in Section II[cite: 1].
3) **Bit-ordering inconsistencies:** mitigated by the frozen conventions specification and automated bidirectional differential tests against the reference implementation[cite: 1].
4) **Numerical drift:** accumulated IEEE-754 rounding over deep circuits; mitigated by per-gate norm verification and selective renormalization at $10^{-9}$ drift threshold[cite: 1].

## References


[1] C. Gidney, "Quirk: A drag-and-drop quantum circuit simulator that runs in your browser," GitHub repository, Apache-2.0 license. [Online]. Available: https://github.com/strilanc/quirk

[2] A. Javadi-Abhari *et al.*, "Qiskit: An open-source framework for quantum computing," IBM, 2017. [Online]. Available: https://qiskit.org

[3] H. De Raedt and K. Michielsen, "Computational methods for simulating quantum computers," in *Quantum Computing and Quantum Bits in Mesoscopic Systems*. Hoboken, NJ, USA: Wiley, 2004.

[4] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, "qHiPSTER: The Quantum High Performance Software Toolkit," 2016, *arXiv:1601.07195*.

[5] A. W. Cross *et al.*, "OpenQASM 3 specification," 2021. [Online]. Available: https://openqasm.com